



Web Technologies

WET2VO

Teil 8: Sessions und Authentifizierung

Medientechnik und -design | SS 2026

HTTP-Response Header

Kopfdaten einer HTTP-Antwort beeinflussen

Response Header



HTTP/1.1 200 OK

Date: Mon, 25 May 2026 13:49:20 GMT

Server: Apache/2.4.67 (Debian)

X-Powered-By: PHP/8.5.6

Vary: Accept-Encoding

Content-Encoding: gzip

Content-Length: 28089

Keep-Alive: timeout=5, max=99

Connection: Keep-Alive

Content-Type: text/html; charset=UTF-8

Header einfügen



```
header("Content-type: text/plain");
```

```
header("Content-type: text/xml");
```

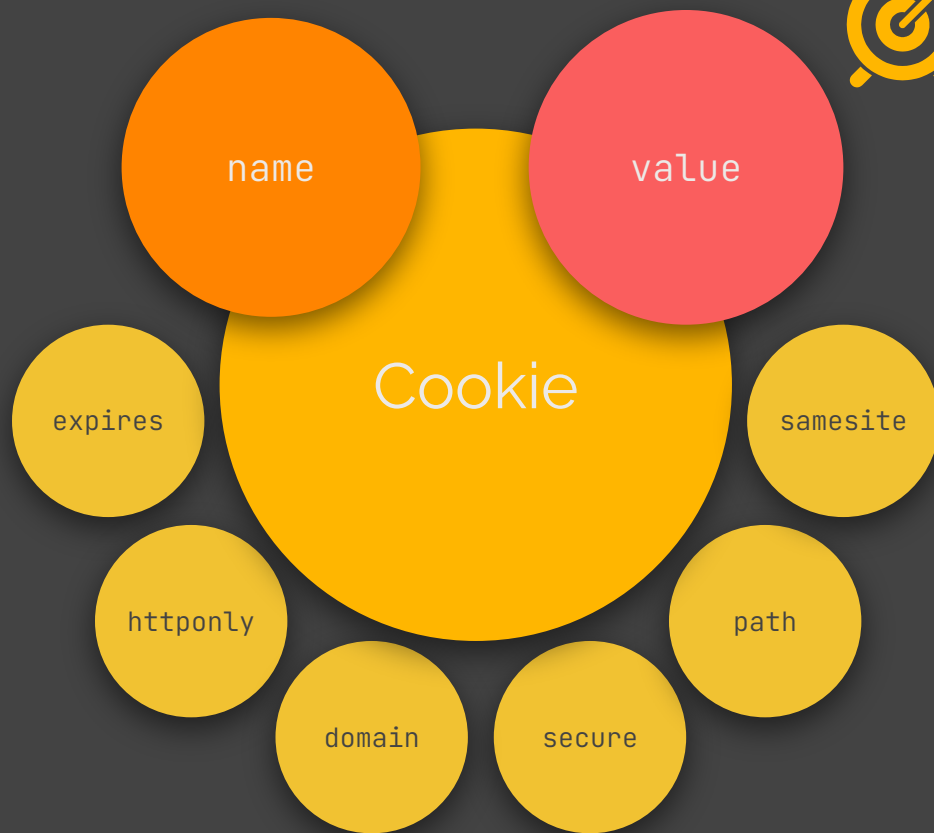
```
header("Location: https://www.fh-ooe.at/");
```

```
exit;
```

Cookies

Daten zwischen Requests austauschen

Parameter eines Cookies



Cookies **setzen**



```
setcookie($name [, $value, [$options]])
```

`$options`: Assoziatives Array mit restlichen Cookie-Parametern.

Einfachster Fall: Session-Cookie

```
setcookie("user", "John Doe");
```

Wird beim Beenden des Browsers gelöscht.

Cookies **erweitert** setzen



Cookie mit Ablaufdatum:

```
setcookie("username", "Jane Doe",  
         ["expires" ⇒ time() + 604800]);
```

Cookie mit Ablaufdatum und anderen Parametern:

```
setcookie("username", "Jane Doe", [  
    "expires" ⇒ time() + 3600, "path" ⇒ "/users/",  
    "domain" ⇒ ".example.com", "secure" ⇒ true,  
    "httponly" ⇒ true, "samesite" ⇒ "Strict"  
]);
```


Auslesen & löschen



Auslesen:

```
echo "<p>Hello {$_COOKIES["username"]}!</p>";
```

Löschen:

```
if (isset($_COOKIE["username"])) {  
    setcookie("username", "",  
              ["expires" => time() - 43200]);  
    unset($_COOKIE["username"]);  
}
```

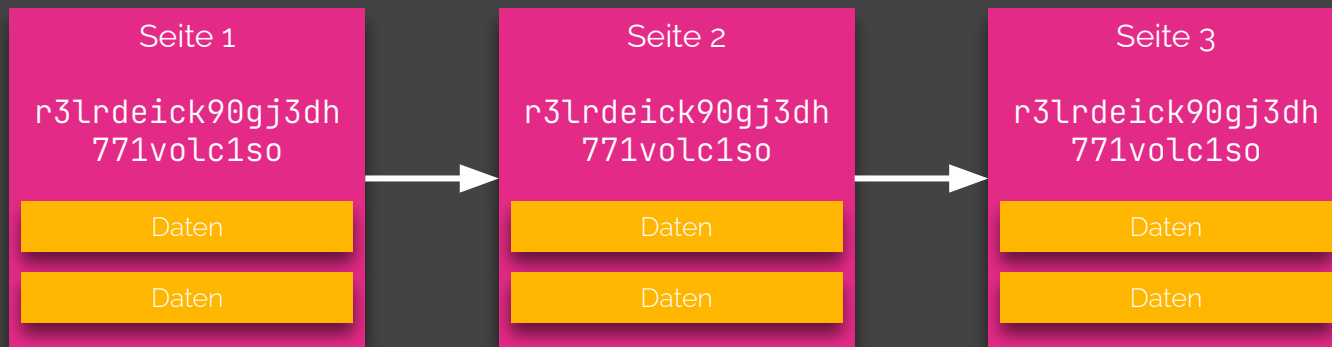
Sessions

Requests als zusammengehörig betrachten



Die Session-ID

z.B. `r3lrdeick90gj3dh771volc1so` (26-stelliger Hex-Wert)



Übertragung mittels **Cookie** (alternativ: URL Rewriting, verstecktes Formularfeld)

`session.use_cookies = 1` (php.ini)



Session-Cookie

name: **PHPSESSID**

value: r3lrdeick90gj3dh771volc1so

lifetime: 0

path: /

domain: ""

secure: false

httponly: false

samesite: ""

Sessions verwenden



```
<?php
session_start();

if (!isset($_SESSION["counter"])) {
    $_SESSION["counter"] = 0;
} else {
    $_SESSION["counter"]++;
}

echo "Anzahl Reloads: {" . $_SESSION["counter"] . "}";
```



Ablauf einer Session





Initialer Request : Kein Cookie

▼ Request Headers ☐ Raw	
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Connection	keep-alive
Host	localhost:8080
Sec-Ch-Ua	"Chromium";v="148", "Google Chrome";v="148", "Not(A)Brand";v="99"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36



Response :

Cookie wird mitgeschickt

▼ Response headers ☐ Raw	
Cache-Control	no-store, no-cache, must-revalidate
Connection	Keep-Alive
Content-Encoding	gzip
Content-Length	198
Content-Type	text/html; charset=UTF-8
Date	Mon, 25 May 2026 13:57:42 GMT
Expires	Thu, 19 Nov 1981 08:52:00 GMT
Keep-Alive	timeout=5, max=100
Pragma	no-cache
Server	Apache/2.4.67 (Debian)
Set-Cookie	PHPSESSID=ac348910cbe76d9c46a8effdb5d1f82b; path=/
Vary	Accept-Encoding
X-Powered-By	PHP/8.5.6



Erneuter Request : Mit Cookie

▼ Request Headers ☐ Raw	
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Connection	keep-alive
Cookie	PHPSESSID=ac348910cbe76d9c46a8effdb5d1f82b
Host	localhost:8080
Referer	http://localhost:8080/wet2vo-examples/vo08/sessions/session_counter.php
Sec-Ch-Ua	"Chromium";v="148", "Google Chrome";v="148", "Not(A)Brand";v="99"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	same-origin
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36

Sessions beenden



1. Session fortführen

```
session_start();
```

2. Session-Array löschen

```
$_SESSION = [];
```

3. Session-Cookie löschen

```
if (isset($_COOKIE[session_name()])) {  
    $params = session_get_cookie_params();  
    setcookie(session_name(), "", [  
        "expires" => time() - 42000, "path" => $params["path"],  
        "domain" => $params["domain"], "secure" => $params["secure"],  
        "httponly" => $params["httponly"], "samesite" => $params["samesite"]  
    ]);  
}
```

4. Session logisch löschen

```
session_destroy();
```

Mehr zu Sessions

Sicherheit und weitere Einstellungen





Die Session-ID
geheim halten.



Die Session-ID regelmäßig **wechseln** .

```
session_regenerate_id();
```




Sessions

benennen .

```
session_name("AwesomeSession1");
```



Status einer **Session abfragen.**

```
session_status();
```

```
PHP_SESSION_DISABLED
```

```
PHP_SESSION_NONE
```

```
PHP_SESSION_ACTIVE
```





Die Session-ID setzen oder abfragen .

```
session_id();
```


Die Session **Einstellungen** in der php.ini überschreiben.

```
session_start(["cookie_lifetime" ⇒ 3600]);
```



Authentifizierung

Mit Sessions Seiten schützen.



Funktionsweise



4 Routen: GET `/login`, POST `/login`, GET `/main`, GET `/logout`

Weiterleitung (forward) mittels `header("Location: ...")` Anweisung



Login -Daten

E-Mail-Adresse und Passwort in einer Datenbank

Schema: `login_example`

Tabelle: `user`

user_id (PK)	email	password
1	user1@email.com	\$2y\$12\$qH2MRZjtJWKujnn7STUXxu0cruWlb7fa.d6k1ZAcKvHFbcAJrQrPm
2	user2@email.com	\$2y\$12\$0/EAkW8Kv/7xLc0myrWz00bgGoWzINJQLDsAhaG2LbbVpNUTIVX.C

Passwörter hashen



```
password_hash($password, $algo);  
password_verify($password, $hash);
```

Immer der aktuell optimale
Algorithmus.
Derzeit: BCRYPT

Beispiel:

```
$password = "geheim";  
$hash = password_hash($password, PASSWORD_DEFAULT);  
echo $hash;  
// $2y$12$qH2MRZjtJWKujnn7STUXXu0cruWlb7fa.d6k1ZAcKvHFbcAJrQrPm  
if (password_verify($password, $hash)) {  
    echo "Passwort korrekt!";  
}
```

Passwörter hashen



`password_verify()` erkennt, mit welchem Algorithmus das Passwort verschlüsselt wurde (Marker).

bcrypt: `$2y$`

`$2y$12$qH2MRZjtJWKujnn7STUXXu0cruWlb7fa.d6k1ZAcKvHFbcAJrQrPm`

Argon2i: `$argon2i$`

`$argon2i$v=19$m=65536,t=4,p=1$ZWM5Wm85bG1sQjVycUJSSQ$PUw89uYkFXlG4AGy2eUe2EMi8yuGEr3ySQF8x2ubr6g`

Argon2id: `$argon2id$`

`$argon2id$v=19$m=65536,t=4,p=1$dkJxZG41dWJQ0VBSdC4zWQ$LPhUBS+abqcK0fnD8uUwk6l4CLbTjqbqYZMkN/RZIj0`

Hash-Upgrades



```
password_needs_rehash($hash, $algo);
```

Beispiel:

```
if (password_verify($password, $hash)) {  
    echo "Passwort korrekt!";  
    if (password_needs_rehash($hash, PASSWORD_DEFAULT)) {  
        $newHash = password_hash($password, PASSWORD_DEFAULT);  
        // Hash in der Datenbank updaten  
    }  
}
```

Hat sich der Wert hinter
`PASSWORD_DEFAULT` geändert oder
wurde z.B. der Kostenfaktor geändert,
wird das Passwort neu gehashed.



Hash- Informationen



```
password_get_info($hash, $algo);
```

Beispiel:

```
$password = "geheim";  
$hash = password_hash($password, PASSWORD_DEFAULT);  
$info = password_get_info($hash);  
var_dump($info);
```

```
array (size=3)  
  'algo' ⇒ string '2y' (length=2)  
  'algoName' ⇒ string 'bcrypt' (length=6)  
  'options' ⇒  
    array (size=1)  
      'cost' ⇒ int 12
```




Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	57 minutes	2 hours	4 hours
6	Instantly	46 minutes	2 days	6 days	2 weeks
7	Instantly	20 hours	4 months	1 year	2 years
8	Instantly	3 weeks	15 years	62 years	164 years
9	2 hours	2 years	791 years	3k years	11k years
10	1 day	40 years	41k years	238k years	803k years
11	1 weeks	1k years	2m years	14m years	56m years
12	3 months	27k years	111m years	917m years	3bn years
13	3 years	705k years	5bn years	56bn years	275bn years
14	28 years	18m years	300bn years	3tn years	19tn years
15	284 years	477m years	15tn years	218tn years	1qd years
16	2k years	12bn years	812tn years	13qd years	94qd years
17	28k years	322bn years	42qd years	840qd years	6qn years
18	284k years	8tn years	2qn years	52qn years	463qn years

**Time it takes
a hacker to
brute force
your password
in 2025**

**Hardware: 12 x RTX 5090
Password hash: bcrypt (10)**



Hive Systems

**Read more and download at
hivesystems.com/password**



Zurück zum Login



4 Routen anlegen



```
$router→get("/login", function () use ($latte, $router) {  
    // Login-Formular anzeigen  
});
```

```
$router→post("/login", function () use ($latte, $router) {  
    // Login-Formular verarbeiten  
});
```

```
$router→get("/main", function () use ($latte, $router) {  
    // Geschützte Hauptseite anzeigen  
});
```

```
$router→get("/logout", function () use ($router) {  
    // Ausloggen  
});
```

GET /login



```
$router→get("/login", function () use ($latte, $router) {  
    $latte→render("login.latte");  
});
```

login.latte:

```
<form method="post" action="{url_for('/login')}">  
...  
</form>
```

POST /login



```
$router->post("/login", function () use ($latte, $router) {  
    $login = new Login($latte, $router);  
    $login->isValid();  
    $login->displayOutput();  
});
```

Klasse Login



```
final class Login
{
    private PDO $pdo;
    private array $messages = [];
    private Engine $latte;
    private Router $router;

    public function __construct(Engine $latte, Router $router)
    {
        $this->latte = $latte;
        $this->router = $router;
        $this->initDB();
    }

    // ...
}
```

Klasse Login: isValid()



```
public function isValid(): void
{
    if (Utilities::isEmptyString($_POST["email"])) {
        $this->messages["email"] = "Please enter your email address.";
    } elseif (!Utilities::isEmail($_POST["email"])) {
        $this->messages["email"] = "Please enter a valid email address";
    }
    if (Utilities::isEmptyString($_POST["password"])) {
        $this->messages["password"] = "Please enter your password.";
    }
    if (count($this->messages) === 0) {
        if (!$this->authenticateUser()) {
            $this->messages["login"] = "Invalid account email address or password.";
        } else {
            $this->business();
        }
    }
}
```

authenticateUser()



```
private function authenticateUser(): bool
{
    $query = "SELECT user_id, email, password FROM user WHERE email = :email";
    $params = [":email" => $_POST["email"]];

    $statement = $this->pdo->prepare($query);
    $statement->execute($params);
    $row = $statement->fetch();

    if ($row && password_verify($_POST["password"], $row["password"])) {
        if (password_needs_rehash($row["password"], PASSWORD_DEFAULT)) {
            $password = password_hash($_POST["password"], PASSWORD_DEFAULT);
            $this->updateUser($row["user_id"], $password);
        }
        return true;
    }
    return false;
}
```


Hash-Upgrade



```
private function authenticateUser(): bool
{
    $query = "SELECT user_id, email, password FROM user WHERE email = :email";
    $params = [":email" => $_POST["email"]];

    $statement = $this->pdo->prepare($query);
    $statement->execute($params);
    $row = $statement->fetch();

    if ($row && password_verify($_POST["password"], $row["password"])) {
        if (password_needs_rehash($row["password"], PASSWORD_DEFAULT)) {
            $password = password_hash($_POST["password"], PASSWORD_DEFAULT);
            $this->updateUser($row["user_id"], $password);
        }
        return true;
    }
    return false;
}
```

Klasse Login: **business()**



```
protected function business(): void
{
    $_SESSION["email"] = $_POST["email"];
    $_SESSION["loginToken"] = Utilities::generateLoginToken();
    $this->router->redirectTo("/main");
}
```

```
public static function generateLoginToken(): string
{
    return bin2hex(random_bytes(32));
}
```

Utilities.php (Hilfsklasse)

```
public function redirectTo(string $p): never
{
    $this->redirect($this->urlFor($p));
}

// header("Location": ...) Aufruf auf volle URL
```

Router.php (foo/router)

Session-Cookie **sichern**



```
session_start([  
    "cookie_secure" ⇒ true,  
    "cookie_httponly" ⇒ true,  
    "cookie_samesite" ⇒ "Strict",  
]);
```

Login: **displayOutput()**



```
public function displayOutput(): void
{
    $this->latte->render("login.latte", [
        "email" => $_POST["email"],
        "messages" => $this->messages,
    ]);
}
```

Routen absichern



```
$router->get("/main", function () use ($latte, $router) {  
    RouteGuard::requireLoggedIn("loginToken", $router, "/login");  
    $latte->render("main.latte");  
});
```

RouteGuard.php:

```
public static function requireLoggedIn(string $tokenName,  
                                       Router $router,  
                                       string $exitRoute): void  
{  
    if (!isset($_SESSION[$tokenName])) {  
        $router->redirectTo($exitRoute);  
    }  
}
```

Routen absichern



```
$router→get("/login", function () use ($latte, $router) {  
    RouteGuard::requireNotLoggedIn("loginToken", $router, "/main");  
    $latte→render("login.latte");  
});
```

RouteGuard.php:

```
public static function requireNotLoggedIn(string $tokenName,  
                                          Router $router,  
                                          string $exitRoute): void  
{  
    if (isset($_SESSION[$tokenName])) {  
        $router→redirectTo($exitRoute);  
    }  
}
```

GET /logout



```
$router→get("/logout", function () use ($router) {  
    $_SESSION = [];  
    if (isset($_COOKIE[session_name()])) {  
        setcookie(session_name(), "", [  
            "expires" ⇒ time() - 86400,  
            "path" ⇒ "/",  
        ]);  
    }  
    session_destroy();  
    $router→redirectTo("/");  
});
```



Nächstes Mal in WET2VO

Datum, Zeit und Internationalisierung

Credits



- Hintergrundbild S. 20: Kristina Flour auf Unsplash: <https://unsplash.com/photos/BcidbyKWquw>
- Hintergrundbild S. 21: Sigmund auf Unsplash: <https://unsplash.com/photos/al4RJ--Mw4l>
- Hintergrundbild S. 22: Kate Trysh auf Unsplash: <https://unsplash.com/photos/WX5jK0BT5JQ>
- Hintergrundbild S. 23: Towfiqu barbhuiya auf Unsplash: <https://unsplash.com/photos/oZuBNC-6E2s>
- Hintergrundbild S. 24: AbsolutVision auf Unsplash: https://unsplash.com/photos/82TpEldo_e4
- Hintergrundbild S. 25: NordWood Themes auf Unsplash: <https://unsplash.com/photos/EZSm8xRinXo>
- Time it takes a hacker to brute force your password in 2025: <https://hivesystems.com/password>